

Advanced

Meet in the Middle

```
#include

using namespace std;
typedef long long ll;
const int maxN = 40;

int N, t[maxN];
ll x, sum, cnt;
unordered_map freq;

int main(){
    scanf("%d %lld", &N, &x);
    for(int i = 0; i < N; i++)
        scanf("%d", &t[i]);
    sort(t, t+N);

    if(N == 1){
        printf("%d\n", x == t[0]);
        return 0;
    }

    freq.reserve(1<<(N/2-1));
    for(int i = 0; i < (1<<(N/2-1)); i++){
        sum = 0;
        for(int j = 0; j < N/2-1; j++)
            if(i&(1<
```

Hamming Distance

```
#include

using namespace std;
const int maxN = 2e4;

int N, ans, b[maxN];

int scanBinary(){
    char c;
    int res = 0;
    while((c = getchar()) != '\n'){
        res <= 1;
        res += (c-'0')&1;
    }
    return res;
}

int main(){
    scanf("%d %d ", &N, &ans);
    for(int i = 0; i < N; i++)
        b[i] = scanBinary();
```

```

for(int i = 0; i < N; i++)
    for(int j = i+1; j < N; j++)
        ans = min(ans, __builtin_popcount(b[i]^b[j]));

printf("%d\n", ans);
}

```

Corner Subgrid Count

```

#include
#pragma GCC target("popcnt")

using namespace std;
typedef long long ll;
const int maxN = 3005;

int N;
ll ans;
bitset B[maxN];

int f(int X){
    return X*(X-1);
}

int main(){
    cin.sync_with_stdio(0);
    cin.tie(0);

    cin >> N;
    for(int i = 0; i < N; i++)
        cin >> B[i];

    for(int i = 0; i < N; i++)
        for(int j = i+1; j < N; j++)
            ans += f((B[i]&B[j]).count());

    cout << (ans>>1);
}

```

Reachable Nodes

```

#include

using namespace std;
const int maxN = 5e4+1;

int N, M, a, b, in[maxN];
bitset ans[maxN];
vector G[maxN];
queue Q;

```

```

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        G[b].push_back(a);
        in[a]++;
    }

    for(int i = 1; i <= N; i++){
        if(in[i] == 0){
            ans[i].set(i);
            Q.push(i);
        }
    }

    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        for(int v : G[u]){
            ans[v] |= ans[u];
            in[v]--;
            if(in[v] == 0){
                ans[v].set(v);
                Q.push(v);
            }
        }
    }

    for(int i = 1; i <= N; i++)
        printf("%d%c", (int) ans[i].count(), (" \n")[i==N]);
}

```

Reachability Queries

```

#include

using namespace std;
const int maxN = 5e4+5;

bool vis[maxN];
int N, M, Q, ds[maxN];
vector order, component;
vector G[maxN], H[maxN], SCC[maxN];
bitset reachable[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]){
        ds[u] += ds[v];
        ds[v] = u;
    } else {
        ds[v] += ds[u];
        ds[u] = v;
    }
    return true;
}

```

```

}

void dfs1(int u){
    vis[u] = true;
    for(int v : G[u])
        if(!vis[v])
            dfs1(v);
    order.push_back(u);
}

void dfs2(int u){
    vis[u] = true;
    component.push_back(u);
    for(int v : H[u])
        if(!vis[v])
            dfs2(v);
}

void dfs3(int u){
    vis[u] = true;
    for(int v : SCC[u]){
        if(!vis[v])
            dfs3(v);
        reachable[u] |= reachable[v];
    }
}

int main(){
    scanf("%d %d %d", &N, &M, &Q);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        H[b].push_back(a);
    }

    fill(vis+1, vis+N+1, false);
    for(int i = 1; i <= N; i++)
        if(!vis[i])
            dfs1(i);

    fill(ds+1, ds+N+1, -1);
    fill(vis+1, vis+N+1, false);
    for(int i = 1; i <= N; i++){
        int u = order[N-i];
        if(!vis[u]){
            dfs2(u);
            for(int j = 0; j+1 < (int) component.size(); j++)
                merge(component[j], component[j+1]);
            component.clear();
        }
    }

    for(int i = 1; i <= N; i++){
        int rep = find(i);
        reachable[rep].set(i);
        for(int v : G[i])
            SCC[rep].push_back(find(v));
    }

    fill(vis+1, vis+N+1, false);
    for(int i = 1; i <= N; i++)
        if(!vis[i])
            dfs3(i);

    for(int i = 0, a, b; i < Q; i++){
        scanf("%d %d", &a, &b);
        a = find(a); b = find(b);
        printf(reachable[a][b] ? "YES\n" : "NO\n");
    }
}

```

```

    }
}

```

Cut and Paste

```

#include

using namespace std;
const int maxN = 2e5+5;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

struct Node {
    char ch;
    int prior, sz;
    Node *l, *r;

    Node(){}
    Node(char c, int p){
        prior = p;
        ch = c;
        sz = 1;
        l = r = nullptr;
    }
};

int N, M, a, b;
char S[maxN];
Node *root;

int sz(Node *t){ return t ? t->sz : 0; }

void pull(Node *t){
    if(!t) return;
    t->sz = sz(t->l) + sz(t->r) + 1;
}

void push(Node *t){
    if(!t) return;
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};

    pair y = {nullptr, nullptr};

```

```

    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(S[x+mid], dist(rng));
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void cut(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    y.second = merge(z.second, z.first);
    root = merge(y.first, y.second);
}

void print(Node *t){
    if(!t) return;
    push(t);
    print(t->l);
    printf("%c", t->ch);
    print(t->r);
}

int main(){
    scanf("%d %d %s", &N, &M, S);
    root = build(0, N);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        int len = (b-a+1);
        cut(a, len);
    }
    print(root);
}

```

Substring Reversals

```
#include

using namespace std;
const int maxN = 2e5+5;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

struct Node {
    bool rev;
    char ch;
    int prior, sz;
    Node *l, *r;

    Node(){}
    Node(char c, int p){
        prior = p;
        ch = c;
        sz = 1;
        l = r = nullptr;
    }
};

int N, M, a, b;
char S[maxN];
Node *root;

int sz(Node *t){ return t ? t->sz : 0; }

void flip(Node *t){
    if(!t) return;
    t->rev ^= true;
}

void pull(Node *t){
    if(!t) return;
    t->sz = sz(t->l) + sz(t->r) + 1;
}

void push(Node *t){
    if(!t) return;
    if(t->rev){
        swap(t->l, t->r);
        flip(t->l); flip(t->r);
        t->rev = false;
    }
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};
```

```

pair y = {nullptr, nullptr};
push(x);
if(k <= sz(x->l)){
    y = split(x->l, k);
    x->l = y.second;
    pull(x);
    y.second = x;
} else {
    y = split(x->r, k-sz(x->l)-1);
    x->r = y.first;
    pull(x);
    y.first = x;
}
return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(S[x+mid], dist(rng));
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void reverse(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    flip(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
}

void print(Node *t){
    if(!t) return;
    push(t);
    print(t->l);
    printf("%c", t->ch);
    print(t->r);
}

int main(){
    scanf("%d %d %s", &N, &M, S);
    root = build(0, N);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        int len = (b-a+1);
        reverse(a, len);
    }
    print(root);
}

```

Reversals and Sums

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

struct Node {
    bool rev;
    int prior, sz;
    ll val, sum;
    Node *l, *r;

    Node(){}
    Node(ll v, int p){
        prior = p;
        sum = val = v;
        sz = 1;
        l = r = nullptr;
    }
};

int N, M, type, a, b;
ll val[maxN];
Node *root;

int sz(Node *t){ return t ? t->sz : 0; }
ll sum(Node *t){ return t ? t->sum : 0; }

void flip(Node *t){
    if(!t) return;
    t->rev ^= true;
}

void pull(Node *t){
    if(!t) return;
    t->sz = sz(t->l) + sz(t->r) + 1;
    t->sum = sum(t->l) + sum(t->r) + t->val;
}

void push(Node *t){
    if(!t) return;
    if(t->rev){
        swap(t->l, t->r);
        flip(t->l); flip(t->r);
        t->rev = false;
    }
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
    }
}
```

```

        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};

    pair y = {nullptr, nullptr};
    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(val[x+mid], dist(rng));
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void reverse(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    flip(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
}

ll getSum(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    ll ans = sum(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
    return ans;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++)
        scanf("%lld", &val[i]);
}

```

```

root = build(0, N);

for(int i = 0; i < M; i++){
    scanf("%d %d %d", &type, &a, &b);
    int len = (b-a+1);
    if(type == 1)
        reverse(a, len);
    else if(type == 2)
        printf("%lld\n", getSum(a, len));
}
}

```

Necessary Roads

```

#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;

int N, M, timer, tin[maxN], low[maxN];
bool vis[maxN];
vector G[maxN];
vector bridges;

void dfs(int u = 1, int p = 0){
    vis[u] = true;
    tin[u] = low[u] = ++timer;
    for(int v : G[u]){
        if(v != p){
            if(vis[v]) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if(low[v] > tin[u])
                    bridges.push_back({u, v});
            }
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    printf("%d\n", (int) bridges.size());
    for(pii P : bridges)
        printf("%d %d\n", P.first, P.second);
}

```

Necessary Cities

```
#include

using namespace std;
typedef pair pii;
const int maxN = 1e5+1;

bool vis[maxN], is_artic[maxN];
int N, M, timer, tin[maxN], low[maxN];
vector ans, G[maxN];

void dfs(int u = 1, int p = -1){
    vis[u] = true;
    tin[u] = low[u] = ++timer;

    int children = 0;
    for(int v : G[u]){
        if(v != p){
            if(vis[v]) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if(low[v] >= tin[u] && p != -1){
                    if(!is_artic[u]){
                        ans.push_back(u);
                        is_artic[u] = true;
                    }
                }
                children++;
            }
        }
    }

    if(p == -1 && children > 1){
        if(!is_artic[u]){
            ans.push_back(u);
            is_artic[u] = true;
        }
    }
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        G[a].push_back(b);
        G[b].push_back(a);
    }

    dfs();

    int K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d%c", ans[i], (" \n")[i==K-1]);
}
```

Eulerian Subgraphs

```

#include

using namespace std;
const int maxN = 1e5+1;
const int MOD = 1e9+7;

int N, M, ans, ds[maxN];

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    ans = 1;
    for(int i = 0, a, b; i < M; i++){
        scanf("%d %d", &a, &b);
        if(!merge(a, b))
            ans = (2 * ans) % MOD;
    }
    printf("%d\n", ans);
}

```

Monster Game I

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;
const int SIZE = 4e6;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll m, b;
    ll operator()(const ll x) const {
        return m * x + b;
    }
} seg[SIZE];

int N, lo[SIZE], hi[SIZE];
ll X, s[maxN], f[maxN];

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    seg[i] = {0, INF};
    if(l == r) return;
    int m = (l+r)/2;

```

```

    build(2*i, l, m);
    build(2*i+1, m+1, r);
}

void insert(int i, Line L){
    int l = lo[i], r = hi[i];
    if(l == r){
        if(L(l) < seg[i](l))
            seg[i] = L;
        return;
    }

    int m = (l+r)/2;
    if(seg[i].m < L.m) swap(seg[i], L);
    if(seg[i](m) > L(m)){
        swap(seg[i], L);
        insert(2*i, L);
    } else insert(2*i+1, L);
}

ll query(int i, ll x){
    int l = lo[i], r = hi[i];
    if(l == r) return seg[i](x);

    int m = (l+r)/2;
    if(x < m) return min(seg[i](x), query(2*i, x));
    else return min(seg[i](x), query(2*i+1, x));
}

int main(){
    scanf("%d %lld", &N, &X);
    for(int i = 0; i < N; i++) scanf("%lld", &s[i]);
    for(int i = 0; i < N; i++) scanf("%lld", &f[i]);

    build(1, 1, 1e6);
    insert(1, {X, 0});
    for(int i = 0; i < N-1; i++){
        ll best = query(1, s[i]);
        insert(1, {f[i], best});
    }
    printf("%lld\n", query(1, s[N-1]));
}

```

Monster Game II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;
const int SIZE = 4e6;
const ll INF = 0x3f3f3f3f3f3f3f3f;

struct Line {
    ll m, b;
    ll operator()(const ll x) const {
        return m * x + b;
    }
} seg[SIZE];

int N, lo[SIZE], hi[SIZE];

```

```

ll X, s[maxN], f[maxN];

void build(int i, int l, int r){
    lo[i] = l; hi[i] = r;
    seg[i] = {0, INF};
    if(l == r) return;
    int m = (l+r)/2;
    build(2*i, l, m);
    build(2*i+1, m+1, r);
}

void insert(int i, Line L){
    int l = lo[i], r = hi[i];
    if(l == r){
        if(L(l) < seg[i](l))
            seg[i] = L;
        return;
    }

    int m = (l+r)/2;
    if(seg[i].m < L.m) swap(seg[i], L);
    if(seg[i](m) > L(m)){
        swap(seg[i], L);
        insert(2*i, L);
    } else insert(2*i+1, L);
}

ll query(int i, ll x){
    int l = lo[i], r = hi[i];
    if(l == r) return seg[i](x);

    int m = (l+r)/2;
    if(x < m) return min(seg[i](x), query(2*i, x));
    else return min(seg[i](x), query(2*i+1, x));
}

int main(){
    scanf("%d %lld", &N, &X);
    for(int i = 0; i < N; i++) scanf("%lld", &s[i]);
    for(int i = 0; i < N; i++) scanf("%lld", &f[i]);

    build(1, 1, 1e6);
    insert(1, {X, 0});
    for(int i = 0; i < N-1; i++){
        ll best = query(1, s[i]);
        insert(1, {f[i], best});
    }
    printf("%lld\n", query(1, s[N-1]));
}

```

Subarray Squares

```

#include

using namespace std;
typedef long long ll;
typedef pair pll;
const int maxN = 3005;

struct Line {
    ll m, b, c;

```

```

    ll operator()(ll x){
        return m * x + b;
    }
};

struct CHT {
    Line dq[2*maxN];
    int fptr, bptr;

    void clear(){
        dq[0] = {0, 0, 0};
        fptr = 0; bptr = 1;
    }

    bool pop_back(Line& L, Line& L1, Line& L2){
        ll v1 = (L.b - L2.b) * (L2.m - L1.m);
        ll v2 = (L2.m - L.m) * (L1.b - L2.b);
        return (v1 == v2 ? L.c > L1.c : v1 < v2);
    }

    bool pop_front(Line& L1, Line& L2, ll x){
        ll v1 = L1(x);
        ll v2 = L2(x);
        return (v1 == v2 ? L1.c < L2.c : v1 > v2);
    }

    void insert(Line L){
        while(bptr-fptr >= 2 && pop_back(L, dq[bptr-1], dq[bptr-2]))    bptr--;
        dq[bptr++] = L;
    }

    pll query(ll x){
        while(bptr-fptr >= 2 && pop_front(dq[fptr], dq[fptr+1], x))    fptr++;
        return {dq[fptr](x), dq[fptr].c};
    }
};

CHT cht;
int N, K, cnt[maxN];
ll pre[maxN], dp[maxN];

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &pre[i]);
        pre[i] += pre[i-1];
        dp[i] = pre[i]*pre[i];
    }

    for(int k = 1; k <= K-1; k++){
        cht.clear();
        for(int i = 1; i <= k; i++)
            cht.insert({-2*pre[i], dp[i]+pre[i]*pre[i], cnt[i]});
        for(int i = k+1; i <= N; i++){
            pll P = cht.query(pre[i]);
            cht.insert({-2*pre[i], dp[i]+pre[i]*pre[i], cnt[i]});
            dp[i] = pre[i]*pre[i] + P.first;
            cnt[i] = P.second + 1;
        }
    }

    printf("%lld\n", dp[N]);
}

```

Houses and Schools

```
#include

using namespace std;
typedef long long ll;
const int maxN = 3001;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, K;
ll best, p[3][maxN], dp[maxN][maxN];

ll travel(int d, int a, int b){
    return (p[d][b] - p[d][a-1]) - (p[0][b] - p[0][a-1]) * (d == 1 ? a : N-b+1);
}

ll C(int a, int b){
    int m = (a+b)/2;
    return travel(1, a, m) + travel(2, m+1, b);
}

void solve(int k, int a = 1, int b = N, int optl = 1, int optr = N){
    if(a > b) return;
    int m = (a+b)/2;
    int opt = -1;
    dp[k][m] = INF;
    for(int i = optl; i <= m; i++){
        if(dp[k-1][i] + C(i, m) < dp[k][m]){
            dp[k][m] = dp[k-1][i] + C(i, m);
            opt = i;
        }
    }
    solve(k, a, m-1, optl, opt);
    solve(k, m+1, b, opt, optr);
}

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= N; i++){
        ll x;
        scanf("%lld", &x);
        p[0][i] = p[0][i-1] + x;
        p[1][i] = p[1][i-1] + i * x;
        p[2][i] = p[2][i-1] + (N-i+1) * x;
    }

    for(int i = 1; i <= N; i++)
        dp[1][i] = travel(2, 1, i);
    for(int k = 2; k <= K; k++)
        solve(k);

    best = INF;
    for(int i = 1; i <= N; i++)
        best = min(best, dp[K][i] + travel(1, i, N));
    printf("%lld\n", best);
}
```

Knuth Division

```

#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f3f;
const int maxN = 5005;

int N, split[maxN][maxN];
ll pre[maxN], dp[maxN][maxN];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf("%lld", &pre[i]);
        pre[i] += pre[i-1];
    }

    for(int len = 0; len <= N; len++){
        for(int l = 0; l+len <= N; l++){
            int r = l+len;
            if(len < 2){
                dp[l][r] = 0;
                split[l][r] = l;
                continue;
            }

            dp[l][r] = INF;
            for(int m = split[l][r-1]; m <= split[l+1][r]; m++){
                ll possible = dp[l][m] + dp[m][r] + pre[r] - pre[l];
                if(possible < dp[l][r]){
                    dp[l][r] = possible;
                    split[l][r] = m;
                }
            }
        }
    }

    printf("%lld\n", dp[0][N]);
}

```

Apples and Bananas

```

#include

using namespace std;
typedef double ld;
typedef complex cd;
const int SIZE = 1<<19;
const ld PI = acos(-1);

int N, M, K;
vector A(SIZE), B(SIZE);

void fft(vector &a, bool inv){
    int N = (int) a.size();

    for(int i = 1, j = 0; i < N; i++){
        int bit = N>>1;
        for(; j&bit; bit >>= 1)
            j ^= bit;
    }
}

```

```

        j ^= bit;

        if(i < j)
            swap(a[i], a[j]);
    }

    for(int len = 2; len <= N; len <= 1){
        ld theta = 2*PI / len * (inv ? -1 : 1);
        cd wlen(cos(theta), sin(theta));
        for(int i = 0; i < N; i += len){
            cd w(1);
            for(int j = 0; j < len / 2; j++){
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if(inv)
        for(cd &z : a)
            z /= N;
}

int main(){
    scanf("%d %d %d", &K, &N, &M);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        A[x] += 1;
    }
    for(int i = 0, x; i < M; i++){
        scanf("%d", &x);
        B[x] += 1;
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < SIZE; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = 2; i <= 2*K; i++)
        printf("%lld%c", llround(A[i].real()), (" \n")[i==2*K]);
}

```

One Bit Positions

```

#include

using namespace std;
typedef double ld;
typedef complex cd;
const int maxN = 2e5+5;
const int SIZE = 1<<19;
const ld PI = acos(-1);

char S[maxN];
vector A(SIZE), B(SIZE);

void fft(vector &a, bool inv){

```

```

int N = (int) a.size();

for(int i = 1, j = 0; i < N; i++){
    int bit = N>>1;
    for(; j&bit; bit >>= 1)
        j ^= bit;
    j ^= bit;

    if(i < j)
        swap(a[i], a[j]);
}

for(int len = 2; len <= N; len <= 1){
    ld theta = 2*PI / len * (inv ? -1 : 1);
    cd wlen(cos(theta), sin(theta));
    for(int i = 0; i < N; i += len){
        cd w(1);
        for(int j = 0; j < len / 2; j++){
            cd u = a[i+j], v = a[i+j+len/2] * w;
            a[i+j] = u + v;
            a[i+j+len/2] = u - v;
            w *= wlen;
        }
    }
}

if(inv)
    for(cd &z : a)
        z /= N;
}

int main(){
    scanf(" %s", S);
    int N = (int) strlen(S);
    for(int i = 0; i < N; i++){
        if(S[i] == '1'){
            A[i] += 1;
            B[N-i-1] += 1;
        }
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < SIZE; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = N; i < 2*N-1; i++)
        printf("%lld%c", llround(A[i].real()), (" \n")[i==2*N-2]);
}

```

Signal Processing

```

#include

using namespace std;
typedef double ld;
typedef complex cd;
const int maxN = 2e5+5;
const int SIZE = 1<<19;
const ld PI = acos(-1);

```

```

int N, M;
vector A(SIZE), B(SIZE);

void fft(vector &a, bool inv){
    int n = (int) a.size();

    for(int i = 1, j = 0; i < n; i++){
        int bit = n>>1;
        for(; j&bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if(i < j)
            swap(a[i], a[j]);
    }

    for(int len = 2; len <= n; len <= 1){
        ld theta = 2*PI / len * (inv ? -1 : 1);
        cd wlen(cos(theta), sin(theta));
        for(int i = 0; i < n; i += len){
            cd w(1);
            for(int j = 0; j < len / 2; j++){
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if(inv)
        for(cd &z : a)
            z /= n;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0, a; i < N; i++){
        scanf("%d", &a);
        A[i] = a;
    }
    for(int i = 0, b; i < M; i++){
        scanf("%d", &b);
        B[M-i-1] = b;
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < SIZE; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = 0; i < N+M-1; i++)
        printf("%lld%c", llround(A[i].real()), (" \n")[i==N+M-2]);
}

```

New Roads Queries

```
#include
```

```

using namespace std;
const int maxN = 4e5+1;

int N, M, Q, x, y, root, idcounter;

struct Node {
    bool rev = false;
    Node *c[2] = {nullptr, nullptr}, *p = nullptr;
    int val, mx, id = ++idcounter;

    Node(int v){
        val = mx = v;
    }

    void push(){
        if(rev){
            swap(c[0], c[1]);
            if(c[0]) c[0]->rev ^= true;
            if(c[1]) c[1]->rev ^= true;
            rev = false;
        }
    }

    void pull(){
        mx = val;
        for(int i = 0; i < 2; i++)
            if(c[i])
                mx = max(mx, c[i]->mx);
    }
} *LCT[maxN];

bool notRoot(Node *t){
    return t->p && (t->p->c[0] == t || t->p->c[1] == t);
}

void rotate(Node *t){
    Node *p = t->p;
    bool b = (p->c[0] == t);
    if((t->p = p->p) && notRoot(p)) t->p->c[(t->p->c[1] == p)] = t;
    if((p->c[!b]=t->c[b])) p->c[!b]->p = p;
    t->c[b] = p;
    p->p = t;
    p->pull();
}

void splay(Node *t){
    while(notRoot(t)){
        Node *p = t->p;
        p->push();
        t->push();
        rotate(t);
    }
    t->push();
    t->pull();
}

Node* access(Node *t){
    Node *last = nullptr;
    for(Node *u = t; u; u = u->p){
        splay(u);
        u->c[1] = last;
        last = u;
    }
    splay(t);
    return last;
}

void evert(Node *t){
    access(t);
}

```

```

    t->rev = true;
}

void link(Node *u, Node *v){
    evert(u);
    u->p = v;
}

void cut(Node *u, Node *v){
    evert(u);
    access(v);
    if(v->c[0]) v->c[0]->p = 0;
    v->c[0] = 0;
    v->pull();
}

Node* path(Node *u, Node *v){
    evert(u);
    access(v);
    return v;
}

Node* LCA(Node *u, Node *v){
    evert(LCT[root]);
    access(u);
    return access(v);
}

bool connected(Node *u, Node *v){
    path(u, v);
    while(v->c[0])
        v = v->c[0];
    return u == v;
}

int main(){
    scanf("%d %d %d", &N, &M, &Q);
    for(int i = 1; i <= N; i++)
        LCT[i] = new Node(0);
    for(int i = 1; i <= M; i++){
        scanf("%d %d", &x, &y);
        LCT[N+i] = new Node(i);
        if(!connected(LCT[x], LCT[y])){
            link(LCT[x], LCT[N+i]);
            link(LCT[y], LCT[N+i]);
        }
    }

    root = 1;
    for(int q = 0; q < Q; q++){
        scanf("%d %d", &x, &y);
        if(connected(LCT[x], LCT[y])){
            Node *p = path(LCT[x], LCT[y]);
            printf("%d\n", p->mx);
        } else printf("-1\n");
    }
}

```

Dynamic Connectivity

```

#include

using namespace std;
typedef pair pii;
typedef array triple;
const int maxN = 1e5+1;
const int SIZE = 3e5+1;
const int INF = 0x3f3f3f3f;

int N, M, K, root, components, del_time[SIZE];
bool used[SIZE];
pii edges[SIZE];
triple ops[maxN];
map> edge_id_map;

struct Node {
    bool rev = false;
    Node *c[2] = {nullptr, nullptr}, *p = nullptr;
    int mn, id;

    Node(int i){
        mn = id = i;
    }

    void push(){
        if(rev){
            swap(c[0], c[1]);
            if(c[0]) c[0]->rev ^= true;
            if(c[1]) c[1]->rev ^= true;
            rev = false;
        }
    }

    void pull(){
        mn = id;
        for(int i = 0; i < 2; i++)
            if(c[i] && del_time[c[i]->mn] < del_time[mn])
                mn = c[i]->mn;
    }
} *LCT[SIZE];

bool notRoot(Node *t){
    return t->p && (t->p->c[0] == t || t->p->c[1] == t);
}

void rotate(Node *t){
    Node *p = t->p;
    bool b = (p->c[0] == t);
    if((t->p = p->p) && notRoot(p)) t->p->c[(t->p->c[1] == p)] = t;
    if((p->c[!b]=t->c[b])) p->c[!b]->p = p;
    t->c[b] = p;
    p->p = t;
    p->push();
}

void splay(Node *t){
    while(notRoot(t)){
        Node *p = t->p;
        p->push();
        t->push();
        rotate(t);
    }
    t->push();
    t->pull();
}

Node* access(Node *t){
    Node *last = nullptr;

```

```

    for(Node *u = t; u; u = u->p){
        splay(u);
        u->c[1] = last;
        last = u;
    }
    splay(t);
    return last;
}

void evert(Node *t){
    access(t);
    t->rev = true;
}

void link(Node *u, Node *v){
    evert(u);
    u->p = v;
}

void cut(Node *u, Node *v){
    evert(u);
    access(v);
    if(v->c[0]) v->c[0]->p = 0;
    v->c[0] = 0;
    v->pull();
}

Node* path(Node *u, Node *v){
    evert(u);
    access(v);
    return v;
}

bool connected(Node *u, Node *v){
    path(u, v);
    while(v->c[0])
        v = v->c[0];
    return u == v;
}

void create_edge(int u, int v){
    int id = edge_id_map[{u, v}].front();
    if(!connected(LCT[u], LCT[v])){
        components--;
        link(LCT[id], LCT[u]);
        link(LCT[id], LCT[v]);
        used[id] = true;
    } else {
        int mn = path(LCT[u], LCT[v])->mn;
        if(del_time[mn] < del_time[id]){
            int cu = edges[mn].first;
            int cv = edges[mn].second;
            cut(LCT[mn], LCT[cu]);
            cut(LCT[mn], LCT[cv]);
            used[mn] = false;
            link(LCT[id], LCT[u]);
            link(LCT[id], LCT[v]);
            used[id] = true;
        }
    }
}

void destroy_edge(int u, int v){
    int id = edge_id_map[{u, v}].front();
    edge_id_map[{u, v}].pop_front();
    if(!used[id]) return;
    cut(LCT[id], LCT[u]);
    cut(LCT[id], LCT[v]);
    used[id] = false;
    components++;
}

```

```

}

int main(){
    scanf("%d %d %d", &N, &M, &K);

    root = 1;
    for(int i = 1; i <= N; i++){
        LCT[i] = new Node(i);
        del_time[i] = INF;
    }

    for(int i = 1, a, b; i <= M; i++){
        scanf("%d %d", &a, &b);
        if(a > b) swap(a, b);
        edges[N+i] = {a, b};
        del_time[N+i] = INF;
        LCT[N+i] = new Node(N+i);
        edge_id_map[{a, b}].push_back(N+i);
    }

    for(int i = 1, t, a, b; i <= K; i++){
        scanf("%d %d %d", &t, &a, &b);
        if(a > b) swap(a, b);
        ops[i] = {t, a, b};
        edges[N+M+i] = {a, b};

        if(t == 2){
            int id = edge_id_map[{a, b}].back();
            del_time[id] = i;
        } else {
            del_time[N+M+i] = INF;
            LCT[N+M+i] = new Node(N+M+i);
            edge_id_map[{a, b}].push_back(N+M+i);
        }
    }

    components = N;
    for(int i = N+1; i <= N+M; i++){
        int u = edges[i].first;
        int v = edges[i].second;
        create_edge(u, v);
    }
    printf("%d ", components);

    for(int i = 1; i <= K; i++){
        int t = ops[i][0];
        int u = ops[i][1];
        int v = ops[i][2];
        if(t == 1) create_edge(u, v);
        else destroy_edge(u, v);
        printf("%d%c", components, (" \n")[i==K]);
    }
}

```

Parcel Delivery

```

#include

using namespace std;
typedef long long ll;
const int maxN = 501, maxM = 1001;

```

```

const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, M, K, p[maxN];
ll d[maxN], cap[maxN][maxN], cost[maxN][maxN];
bool inq[maxN];
vector G[maxN];

struct Edge {
    int u, v;
    ll r, c;
} edges[maxM], redges[maxM];

void bellman_ford(){
    fill(inq+1, inq+N+1, false);
    fill(d+1, d+N+1, INF);
    fill(p+1, p+N+1, 0);

    queue Q;
    Q.push(1);
    d[1] = 0;
    inq[1] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

ll minimum_cost_flow(){
    ll flow = 0, cost = 0;
    while(flow < K){
        bellman_ford();
        if(d[N] == INF) break;

        ll aug = K-flow;
        int u = N;
        while(u != 1){
            Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
            aug = min(aug, e.r);
            u = e.u;
        }

        flow += aug;
        cost += aug * d[N];
        u = N;
        while(u != 1){
            if(p[u] < 0){
                redges[-p[u]].r -= aug;
                edges[-p[u]].r += aug;
            } else {
                redges[p[u]].r += aug;
                edges[p[u]].r -= aug;
            }
            u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
        }
    }
    return (flow < K ? -1 : cost);
}

```

```

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 1, u, v, r, c; i <= M; i++){
        scanf("%d %d %d %d", &u, &v, &r, &c);
        G[u].push_back(i);
        G[v].push_back(-i);
        edges[i] = {u, v, r, c};
        redges[i] = {v, u, 0, -c};
    }
    printf("%lld\n", minimum_cost_flow());
}

```

Task Assignment

```

#include

using namespace std;
typedef long long ll;
const int maxN = 402, maxM = 40401;
const ll INF = 0x3f3f3f3f3f3f3f3f;

int N, p[maxN];
ll d[maxN], cap[maxN][maxN], cost[maxN][maxN];
bool inq[maxN], vis[maxM];
vector path, G[maxN];

struct Edge {
    int u, v;
    ll r, c;
} edges[maxM], redges[maxM];

void bellman_ford(int start = 0){
    fill(inq, inq+maxN, false);
    fill(d, d+maxN, INF);
    fill(p, p+maxN, 0);

    queue Q;
    Q.push(start);
    d[start] = 0;
    inq[start] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

ll minimum_cost_flow(){
    ll flow = 0, cost = 0;
    while(flow < N){

```

```

    bellman_ford();
    if(d[2*N+1] == INF) break;

    ll aug = N-flow;
    int u = 2*N+1;
    while(u != 0){
        Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
        aug = min(aug, e.r);
        u = e.u;
    }

    flow += aug;
    cost += aug * d[2*N+1];
    u = 2*N+1;
    while(u != 0){
        if(p[u] < 0){
            redges[-p[u]].r -= aug;
            edges[-p[u]].r += aug;
        } else {
            redges[p[u]].r += aug;
            edges[p[u]].r -= aug;
        }
        u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
    }
}
return (flow < N ? -1 : cost);
}

void dfs(int u = 0){
    if(u == 2*N+1) return;
    if(u != 0) path.push_back(u);
    for(int i : G[u]){
        if(i > 0 && edges[i].r == 0 && !vis[i]){
            vis[i] = true;
            dfs(edges[i].v);
            return;
        }
    }
}

int main(){
    scanf("%d", &N);
    int edgeID = 1;
    for(int u = 1; u <= N; u++){
        for(int v = N+1, c; v <= 2*N; v++){
            scanf("%d", &c);
            G[u].push_back(edgeID);
            G[v].push_back(-edgeID);
            edges[edgeID] = {u, v, 1, c};
            redges[edgeID] = {v, u, 0, -c};
            edgeID++;
        }
    }
    for(int v = 1; v <= N; v++){
        G[0].push_back(edgeID);
        G[v].push_back(-edgeID);
        edges[edgeID] = {0, v, 1, 0};
        redges[edgeID] = {v, 0, 0, 0};
        edgeID++;
    }
    for(int u = N+1; u <= 2*N; u++){
        G[u].push_back(edgeID);
        G[2*N+1].push_back(-edgeID);
        edges[edgeID] = {u, 2*N+1, 1, 0};
        redges[edgeID] = {2*N+1, u, 0, 0};
        edgeID++;
    }

    printf("%lld\n", minimum_cost_flow());
    for(int i = 0; i < N; i++){

```

```

        path.clear();
        dfs();
        printf("%d %d\n", path[0], path[1]-N);
    }
}

```

Distinct Routes II

```

#include

using namespace std;
const int maxN = 501, maxM = 1001;
const int INF = 0x3f3f3f3f;

int N, M, K, p[maxN], d[maxN];
bool inq[maxN], vis[maxM];
vector path, G[maxN];

struct Edge {
    int u, v, r, c;
} edges[maxM], redges[maxM];

void bellman_ford(){
    fill(inq+1, inq+N+1, false);
    fill(d+1, d+N+1, INF);
    fill(p+1, p+N+1, 0);

    queue Q;
    Q.push(1);
    d[1] = 0;
    inq[1] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

int minimum_cost_flow(){
    int flow = 0, cost = 0;
    while(flow < K){
        bellman_ford();
        if(d[N] == INF) break;

        int aug = K-flow;
        int u = N;
        while(u != 1){
            Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
            aug = min(aug, e.r);
            u = e.u;
        }
    }
}

```

```

    }

    flow += aug;
    cost += aug * d[N];
    u = N;
    while(u != 1){
        if(p[u] < 0){
            redges[-p[u]].r -= aug;
            edges[-p[u]].r += aug;
        } else {
            redges[p[u]].r += aug;
            edges[p[u]].r -= aug;
        }
        u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
    }
}
return (flow < K ? -1 : cost);
}

void dfs(int u = 1){
    path.push_back(u);
    if(u == N) return;
    for(int i : G[u]){
        if(i > 0 && edges[i].r == 0 && !vis[i]){
            vis[i] = true;
            dfs(edges[i].v);
            return;
        }
    }
}

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 1, u, v; i <= M; i++){
        scanf("%d %d", &u, &v);
        G[u].push_back(i);
        G[v].push_back(-i);
        edges[i] = {u, v, 1, 1};
        redges[i] = {v, u, 0, -1};
    }

    int minCoins = minimum_cost_flow();
    if(minCoins == -1){
        printf("-1\n");
        return 0;
    }

    printf("%d\n", minCoins);
    for(int i = 0; i < K; i++){
        path.clear();
        dfs();

        int sz = (int) path.size();
        printf("%d\n", sz);
        for(int j = 0; j < sz; j++)
            printf("%d%c", path[j], (" \n")[j==sz-1]);
    }
}

```